Lesson 8: Sound and game states

Pygame CE • S3 onwards • 50-60 min

Learning intention

Use sound and a state variable to manage different parts of a game.

Success criteria

- I can play a sound effect.
- I can use a state variable.
- I can restart the game.

Key ideas

- Sound effects are short **.wav** files loaded with `pygame.mixer.Sound`.
- A **state** variable (such as "playing" or "game_over") decides which code runs each frame.
- Resetting means putting *every* changing value back to its starting value.

Activities

1. Add a collect sound.
2. Draw a state diagram on paper first: playing -> game_over -> playing (press R).
3. Switch to the game-over screen when time runs out.
4. Add a restart key.

Code

Complete program (requires assets/collect.wav)

```
import random
import pygame

pygame.init()

WIDTH = 800
HEIGHT = 600
SIZE = 40
ROUND_TIME = 30

screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Collect game")
clock = pygame.time.Clock()
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 72)

# Needs assets/collect.wav
collect_sound = pygame.mixer.Sound("assets/collect.wav")


def random_target():
    return pygame.Rect(random.randint(0, WIDTH - SIZE),
                       random.randint(0, HEIGHT - SIZE), SIZE, SIZE)


def draw_text(font, message, position, colour=(255, 255, 255)):
    screen.blit(font.render(message, True, colour), position)


def reset():
    return 380.0, 280.0, 0, random_target(), pygame.time.get_ticks(), "playing"
```

```
x, y, score, target, start_ticks, state = reset()
speed = 250
running = True

while running:
    dt = clock.tick(60) / 1000

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r and state == "game_over":
                x, y, score, target, start_ticks, state = reset()

    if state == "playing":
        keys = pygame.key.get_pressed()
        x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * speed * dt
        y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * speed * dt
        x = max(0, min(x, WIDTH - SIZE))
        y = max(0, min(y, HEIGHT - SIZE))

        elapsed = (pygame.time.get_ticks() - start_ticks) / 1000
        remaining = max(0, ROUND_TIME - elapsed)
        if remaining == 0:
            state = "game_over"

        player_rect = pygame.Rect(int(x), int(y), SIZE, SIZE)
        if player_rect.colliderect(target):
            score += 1
            collect_sound.play()
            target = random_target()

    screen.fill((30, 30, 40))
    if state == "playing":
        pygame.draw.rect(screen, (240, 190, 60), target)
        pygame.draw.rect(screen, (80, 200, 120), player_rect)
        draw_text(font, f"Score: {score}", (10, 10))
        draw_text(font, f"Time: {remaining:.0f}", (10, 45))
    else:
        draw_text(big_font, "Game over!", (260, 220))
        draw_text(font, f"Final score: {score}", (300, 300))
        draw_text(font, "Press R to play again", (270, 340))
    pygame.display.flip()

pygame.quit()
```

Checkpoint

A sound plays on collection; the game shows a game-over screen and restarts with R.

Support and stretch

- **Support:** Give the state diagram partly completed, and the reset function prewritten.
- **Stretch:** Add a start screen, or a win state when a target score is reached.

Reflect

List every value that needs resetting for a new game. Did you miss any?

Save your work
Save this version as `lesson8.py` before you start the next lesson.